LangChain & LangGraph

Weeks 3 & 4 -- Day-by-Day Study Plan

Week 3: LangChain Advanced		Week 4: LangGraph Intro	
Day 15	LangChain Tools -- @tool decorator, bind_tools, manual execution loop	Day 22	Why LangGraph -- graphs vs chains, state machines for AI workflows
Day 16	Agents -- create_tool_calling_agent, AgentExecutor, ReAct loop	Day 23	StateGraph -- nodes, edges, TypedDict state, conditional edges
Day 17	LangSmith -- tracing, debugging, evaluation datasets	Day 24	ReAct Agent in LangGraph -- agent node, tool node, looping graph
Day 18	Advanced Retrieval -- HyDE, ParentDocumentRetriever, EnsembleRetriever	Day 25	Checkpointing -- MemorySaver, SqliteSaver, thread IDs
Day 19	Chains & Routing -- RunnableBranch, RunnableParallel, MapReduce	Day 26	Human-in-the-Loop -- interrupt_before, Command, update_state
Day 20-21	Week 3 Project -- Research Assistant with streaming + routing	Day 27-28	Week 4 Project -- Stateful agent with persistence & streaming

PREREQUISITES

Before starting Week 3 confirm you have completed:

Week 1: OpenAI API, embeddings, RAG pipeline, agents concept

Week 2: LCEL chains, PromptTemplates, memory, vector stores, full RAG app

langchain, langchain-openai, langchain-community, chromadb installed

A working RAG app from Days 13-14 to build on in Week 3

Week 3 -- LangChain Advanced

Day 15 . LangChain Tools -- Custom Tools & Tool Calling

(1) THEORY - TOPICS TO COVER

@tool decorator -- auto-generates JSON schema from function docstring
Tool schema: name, description, args -- why descriptions matter so much
bind_tools() on a ChatModel -- attaching tools to an LLM
Parsing ToolCall objects from AIMessage responses
Building a manual tool-execution loop without AgentExecutor

(2) FOCUS / SKIP GUIDE

FOCUS ON

How @tool reads the docstring to build the JSON schema -- write clear docstrings
Why the tool description is what the model reads to decide when to call it
The structure of an AIMessage when the model wants to call a tool
How to extract tool name and args from a ToolCall object and run it yourself
The manual loop pattern: LLM decides -> code runs -> result fed back -> repeat

DON'T WORRY ABOUT YET

AgentExecutor automatic looping -- you do it manually today, executor on Day 16
OpenAPI spec-based tool generation -- niche advanced use case
Async tool calling with ainvoke -- save for production deployment
Tool input validation with Pydantic args schema -- come back after basics

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangChain tools concept docs
-> https://python.langchain.com/docs/concepts/tools/
[Docs] LangChain -- How to create custom tools
-> https://python.langchain.com/docs/how_to/custom_tools/
[YouTube] Sam Witteveen -- Custom tools in LangChain
-> https://www.youtube.com/watch?v=Yw_1ysQ_-VM
[Docs] LangChain -- bind_tools and tool calling
-> https://python.langchain.com/docs/how_to/tool_calling/
[Blog] LangChain blog -- Tool calling best practices
-> https://blog.langchain.dev/tool-calling-with-langchain/

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Create a calculator tool with @tool -- takes a math expression string
- [ ] Create a get_current_time tool with @tool
- [ ] Create a search_wikipedia tool (use wikipedia-api or mock it)
- [ ] Bind all 3 tools to the LLM with bind_tools()
- [ ] Invoke with a query that triggers a tool call -- print the raw AIMessage
- [ ] Manually parse the ToolCall object: extract name and args
- [ ] Build manual loop: LLM -> parse tool call -> run tool -> feed result back -> final answer

TASK RESOURCES

[Docs] LangChain -- @tool decorator API reference
-> https://python.langchain.com/api_reference/core/tools/langchain_core.tools.convert.tool.html
[Docs] LangChain -- How to pass tool results back to model
-> https://python.langchain.com/docs/how_to/tool_results_pass_to_model/
[PyPI] wikipedia-api package
-> https://pypi.org/project/Wikipedia-API/

(4) CODE EXAMPLES

Define tools with @tool decorator

```
from langchain_core.tools import tool
from datetime import datetime
import math

@tool
def calculator(expression: str) -> str:
    '''Evaluate a mathematical expression like '2+2' or 'sqrt(16)'.'''
    try:
        return str(eval(expression, {'__builtins__': {}}, vars(math)))
    except Exception as e:
        return f'Error: {e}'

@tool
def get_current_time() -> str:
    '''Returns the current date and time. Use when asked about time.'''
    return datetime.now().strftime('%Y-%m-%d %H:%M:%S')

@tool
def search_wikipedia(query: str) -> str:
    '''Search Wikipedia for a topic and return a brief summary.'''
    return f'Wikipedia result for "{query}": [summary here]'

tools = [calculator, get_current_time, search_wikipedia]
print(calculator.name)        # 'calculator'
print(calculator.description) # from docstring
print(calculator.args)        # {'expression': {'type': 'string'}}
```

bind_tools + parse ToolCall manually

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, ToolMessage

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
llm_with_tools = llm.bind_tools(tools)

# LLM decides to call a tool
response = llm_with_tools.invoke([HumanMessage('What is 144 * 12?')])
print(response.tool_calls)
# [{'name': 'calculator', 'args': {'expression': '144*12'}, 'id': '...'}]

# Parse and execute manually
tool_map = {t.name: t for t in tools}
messages = [HumanMessage('What is 144 * 12?'), response]
for tc in response.tool_calls:
    result = tool_map[tc['name']].invoke(tc['args'])
    messages.append(ToolMessage(content=str(result), tool_call_id=tc['id']))

final = llm_with_tools.invoke(messages)
print(final.content)
```

Manual agent loop (no AgentExecutor)

```
def run_manual_agent(user_input: str, max_steps: int = 5):
    messages = [HumanMessage(user_input)]
    for step in range(max_steps):
        response = llm_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return response.content
        for tc in response.tool_calls:
            result = tool_map[tc['name']].invoke(tc['args'])
            print(f'  Called {tc["name"]}({tc["args"]}) = {result}')
            messages.append(ToolMessage(content=str(result), tool_call_id=tc['id']))
    return 'Max steps reached'

ans = run_manual_agent('What is sqrt(256) and what time is it now?')
print(ans)
```

READ MORE - TOPIC REFERENCES

[Docs] LangChain -- How to handle tool errors
-> https://python.langchain.com/docs/how_to/tools_error/
[Docs] LangChain -- Tool artifacts (non-string results)
-> https://python.langchain.com/docs/how_to/tool_artifacts/
[Docs] OpenAI -- Function calling deep dive
-> https://platform.openai.com/docs/guides/function-calling

(5) DAY COMPLETION CHECKLIST

- [ ] 3 tools created with @tool -- docstrings are clear and descriptive
- [ ] bind_tools() called -- tool_calls appear in AIMessage for relevant queries
- [ ] Manually parsed a ToolCall: extracted name and args correctly
- [ ] Manual loop runs end-to-end: LLM -> tool -> LLM -> final answer
- [ ] Tested with a 2-step query (e.g. sqrt AND current time) -- both tools called

- [ ] Can explain: why tool descriptions affect whether the model calls them

Day 16 . Agents with LangChain -- AgentExecutor & ReAct

(1) THEORY - TOPICS TO COVER

AgentExecutor -- the loop wrapper that automates tool calling
create_tool_calling_agent -- the modern recommended approach
Agent scratchpad: MessagesPlaceholder tracking intermediate steps
Stopping conditions: max_iterations, early_stopping_method
Verbose output -- reading every intermediate reasoning step

(2) FOCUS / SKIP GUIDE

FOCUS ON

How AgentExecutor automates the manual loop you built on Day 15
The 4-part prompt structure: system, chat_history, input, agent_scratchpad
How verbose=True reveals the Thought -> Action -> Observation cycle
max_iterations preventing infinite loops on tricky queries
handle_parsing_errors=True -- what happens without it vs with it

DON'T WORRY ABOUT YET

Legacy AgentType.ZERO_SHOT_REACT_DESCRIPTION -- deprecated, ignore
Custom agent classes from BaseMultiActionAgent -- not needed
LangGraph-based agents -- Weeks 4-5, don't jump ahead
Agent memory with external databases -- production concern

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangChain agents tutorial (official)
-> https://python.langchain.com/docs/tutorials/agents/
[Docs] LangChain -- create_tool_calling_agent
-> https://python.langchain.com/docs/how_to/agent_executor/
[YouTube] Greg Kamradt -- Build a LangChain agent
-> https://www.youtube.com/watch?v=MO3_Xv5fLzk
[Docs] LangChain -- AgentExecutor reference
-> https://python.langchain.com/api_reference/langchain/agents/langchain.agents.agent.AgentExecutor.html

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Build agent with create_tool_calling_agent using Day 15 tools
- [ ] Wrap in AgentExecutor with verbose=True
- [ ] Test with a simple single-tool query
- [ ] Test with a complex multi-step query requiring 2+ tools
- [ ] Read the verbose trace -- identify Thought, Action, Observation lines
- [ ] Set max_iterations=3 -- observe what happens with a complex query
- [ ] Add handle_parsing_errors=True and test with an ambiguous query

TASK RESOURCES

[Docs] LangChain -- AgentExecutor config options
-> https://python.langchain.com/api_reference/langchain/agents/langchain.agents.agent.AgentExecutor.html
[Docs] LangChain -- How to add memory to agents
-> https://python.langchain.com/docs/how_to/migrate_agent_memory/
[Docs] LangChain -- How to stream from agents
-> https://python.langchain.com/docs/how_to/streaming_agent/

(4) CODE EXAMPLES

create_tool_calling_agent + AgentExecutor

```
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# Required 4-part prompt structure
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant. Use tools when needed.'),
    MessagesPlaceholder('chat_history', optional=True),
    ('human', '{input}'),
    MessagesPlaceholder('agent_scratchpad'),
])

agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(
    agent=agent, tools=tools,
    verbose=True,
    max_iterations=10,
    handle_parsing_errors=True,
)

result = executor.invoke({'input': 'What is 256/16 and what time is it?'})
print(result['output'])
```

Agent with conversation memory

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

store = {}
def get_history(session_id):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

agent_with_memory = RunnableWithMessageHistory(
    executor, get_history,
    input_messages_key='input',
    history_messages_key='chat_history',
)
cfg = {'configurable': {'session_id': 'demo'}}
agent_with_memory.invoke({'input': 'My name is Arjun'}, config=cfg)
agent_with_memory.invoke({'input': 'What is my name?'}, config=cfg)
```

READ MORE - TOPIC REFERENCES

[Docs] LangChain -- How to return structured output from agents
-> https://python.langchain.com/docs/how_to/agent_structured/
[Paper] ReAct: Synergizing Reasoning + Acting (Yao et al. 2022)
-> https://arxiv.org/abs/2210.03629

(5) DAY COMPLETION CHECKLIST

- [ ] AgentExecutor built with create_tool_calling_agent -- runs without error
- [ ] verbose=True -- Thought / Action / Observation cycle visible in terminal
- [ ] Single-tool query works correctly
- [ ] Multi-step query uses 2+ tools in sequence correctly
- [ ] max_iterations tested -- agent stops gracefully when limit hit
- [ ] Memory added with RunnableWithMessageHistory -- agent remembers context
- [ ] Can explain: when to use AgentExecutor vs the Day 15 manual loop

---

Day 17 . LangSmith -- Tracing, Debugging & Evaluation

(1) THEORY - TOPICS TO COVER

What LangSmith does: traces every LLM call, token usage, and latency
Runs, traces, spans -- how to read the LangSmith trace hierarchy
Latency analysis: finding the slowest step in your chain
Datasets: creating Q&A; pairs for repeatable automated evaluation
Evaluators: LLM-as-judge for correctness and relevance scoring

(2) FOCUS / SKIP GUIDE

FOCUS ON

Setting 4 env variables -- that is all it takes to enable full tracing
Reading a trace in the UI: expanding spans to see prompts and raw outputs
Finding bottlenecks: which step uses the most tokens or the most time
Creating a dataset of 5 Q&A; pairs for your existing RAG app
Running an evaluation and understanding what the scores mean

DON'T WORRY ABOUT YET

Custom evaluator functions from scratch -- start with LLM-as-judge built-in
LangSmith CI/CD in GitHub Actions -- Week 8 topic
Online production monitoring and alerting setup
LangSmith SDK for programmatic dataset management at scale

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangSmith quickstart docs
-> https://docs.smith.langchain.com/
[YouTube] LangChain -- LangSmith tutorial (official video)
-> https://www.youtube.com/watch?v=Hab2CV_0hpQ
[Docs] LangSmith -- How to create datasets
-> https://docs.smith.langchain.com/how_to_guides/datasets
[Docs] LangSmith -- Evaluation how-to
-> https://docs.smith.langchain.com/how_to_guides/evaluation
[Blog] LangChain blog -- LLM evaluation best practices
-> https://blog.smith.langchain.com/llm-eval-best-practices

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Sign up for LangSmith free tier at smith.langchain.com
- [ ] Add 4 env vars to .env: LANGCHAIN_TRACING_V2, API_KEY, PROJECT, ENDPOINT
- [ ] Run your Week 2 RAG app -- verify traces appear in LangSmith UI
- [ ] Click into a trace -- expand all spans, find the LLM call span
- [ ] Identify the slowest step and the step using the most tokens
- [ ] Create a dataset with 5 Q&A; pairs for your RAG app in the UI
- [ ] Run a basic LLM-as-judge evaluation on those 5 pairs

TASK RESOURCES

[Docs] LangSmith -- Environment variables setup
-> https://docs.smith.langchain.com/how_to_guides/setup/
[Docs] LangSmith -- Running evaluations
-> https://docs.smith.langchain.com/how_to_guides/evaluation/evaluate_llm_application
[Docs] LangSmith -- LLM-as-judge evaluator
-> https://docs.smith.langchain.com/how_to_guides/evaluation/evaluate_llm_application#use-llm-as-judge

(4) CODE EXAMPLES

LangSmith setup -- add to .env

```
# Add these 4 lines to your .env file:
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls__your_key_here
LANGCHAIN_PROJECT=my-study-project
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com

# That is it -- all LangChain calls are now traced automatically.
# Run any chain or agent then check: https://smith.langchain.com
```

Create dataset + run evaluation

```
from langsmith import Client
from langsmith.evaluation import evaluate, LangChainStringEvaluator

client = Client()

# Create dataset
dataset = client.create_dataset('rag-eval-v1')
examples = [
    {'inputs': {'question': 'What is RAG?'},
     'outputs': {'answer': 'RAG stands for Retrieval-Augmented Generation...'}},
    {'inputs': {'question': 'What is chunking?'},
     'outputs': {'answer': 'Chunking splits documents into smaller pieces...'}},
]
client.create_examples(dataset_id=dataset.id, examples=examples)

# Define the pipeline under test
def rag_pipeline(inputs):
    return {'answer': rag_chain.invoke({'input': inputs['question']})['answer']}

# Run evaluation
evaluator = LangChainStringEvaluator('qa', config={'llm': llm})
results = evaluate(rag_pipeline, data='rag-eval-v1', evaluators=[evaluator])
print(results.to_pandas())
```

READ MORE - TOPIC REFERENCES

[Docs] LangSmith -- Annotation queues for human review
-> https://docs.smith.langchain.com/how_to_guides/human_feedback
[Blog] LangChain -- Evaluating RAG with LangSmith
-> https://blog.smith.langchain.com/evaluating-rag-pipelines-with-langsmith

(5) DAY COMPLETION CHECKLIST

- [ ] LangSmith free tier account created
- [ ] All 4 env variables set -- LANGCHAIN_TRACING_V2=true confirmed
- [ ] RAG app run -- traces visible in LangSmith UI
- [ ] Trace expanded -- found LLM call span with prompt and output
- [ ] Identified slowest step and highest token-usage step
- [ ] Dataset with 5 Q&A; pairs created in LangSmith
- [ ] LLM-as-judge evaluation ran -- scores visible in UI
- [ ] Can explain: what is a trace, what is a span, what is an evaluator

---

Day 18 . Advanced Retrieval Patterns

(1) THEORY - TOPICS TO COVER

Hypothetical Document Embeddings (HyDE) -- generate then embed then retrieve
ParentDocumentRetriever -- embed small chunks, return large parent chunks
SelfQueryRetriever -- natural language metadata filtering
EnsembleRetriever -- combining dense (semantic) + sparse (BM25) search
MultiVectorRetriever -- multiple representations per document

(2) FOCUS / SKIP GUIDE

FOCUS ON

- Why naive chunking loses context -- how ParentDocumentRetriever fixes this
- HyDE: embed a hypothetical answer instead of the raw question
- SelfQueryRetriever: converts 'show me pages from chapter 3' to a filter
- BM25 catches exact keyword matches that semantic search misses
- How to A/B test two retrievers on your 20-question set from Week 2

DON'T WORRY ABOUT YET

- ColBERT and learned sparse retrieval -- research-level complexity
- Fine-tuning embedding models on domain data
- Vector index optimisation (HNSW parameters, IVF clustering)
- Re-ranking with cross-encoders -- valid but save for later

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangChain -- ParentDocumentRetriever
-> https://python.langchain.com/docs/how_to/parent_document_retriever/
[Docs] LangChain -- SelfQueryRetriever
-> https://python.langchain.com/docs/how_to/self_query/
[Docs] LangChain -- EnsembleRetriever with BM25
-> https://python.langchain.com/docs/how_to/ensemble_retriever/
[Paper] HyDE paper -- Precise Zero-Shot Dense Retrieval
-> https://arxiv.org/abs/2212.10496
[YouTube] LangChain -- Advanced RAG patterns overview
-> https://www.youtube.com/watch?v=sVcwVQRHIc8
[Docs] LangChain advanced retrieval how-tos index
-> https://python.langchain.com/docs/how_to/#retrievers

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Implement ParentDocumentRetriever -- compare quality to naive chunking
- [ ] Try SelfQueryRetriever with 2 metadata filter types (page, source)
- [ ] Build EnsembleRetriever combining Chroma (dense) + BM25 (sparse)
- [ ] Implement HyDE: generate hypothetical doc -> embed -> retrieve
- [ ] A/B test ParentDoc vs naive on your 20-question set -- log results

TASK RESOURCES

[Docs] LangChain -- MultiVectorRetriever
-> https://python.langchain.com/docs/how_to/multi_vector/
[Docs] LangChain -- BM25Retriever
-> https://python.langchain.com/docs/integrations/retrievers/bm25/
[PyPI] rank-bm25 package
-> https://pypi.org/project/rank-bm25/

(4) CODE EXAMPLES

ParentDocumentRetriever

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_text_splitters import RecursiveCharacterTextSplitter

child_splitter  = RecursiveCharacterTextSplitter(chunk_size=200)
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1000)

from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
vectorstore = Chroma(embedding_function=OpenAIEmbeddings())
docstore    = InMemoryStore()

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore, docstore=docstore,
    child_splitter=child_splitter, parent_splitter=parent_splitter,
)
retriever.add_documents(docs)
results = retriever.invoke('What is the main argument?')
print(f'Got {len(results)} parent chunks')
```

EnsembleRetriever (dense + BM25)

```
from langchain.retrievers import BM25Retriever, EnsembleRetriever

bm25 = BM25Retriever.from_documents(docs)
bm25.k = 4
chroma_ret = vectorstore.as_retriever(search_kwargs={'k': 4})

ensemble = EnsembleRetriever(
    retrievers=[bm25, chroma_ret], weights=[0.5, 0.5]
)
results = ensemble.invoke('exact technical term from document')
```

HyDE -- Hypothetical Document Embeddings

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

hyde_prompt = ChatPromptTemplate.from_template(
    'Write a 2-sentence expert answer to: {question}'
)
hyde_chain = hyde_prompt | llm | StrOutputParser()

def hyde_retrieve(question):
    hypothetical = hyde_chain.invoke({'question': question})
    print(f'Hypothetical: {hypothetical[:80]}...')
    return chroma_ret.invoke(hypothetical)

results = hyde_retrieve('How does the attention mechanism work?')
```

READ MORE - TOPIC REFERENCES

[Docs] LangChain -- Contextual compression retriever
-> https://python.langchain.com/docs/how_to/contextual_compression/
[Blog] Pinecone -- Advanced RAG techniques guide
-> https://www.pinecone.io/learn/advanced-rag/
[Blog] Anthropic -- Contextual retrieval (2024)
-> https://www.anthropic.com/news/contextual-retrieval

(5) DAY COMPLETION CHECKLIST

- [ ] ParentDocumentRetriever set up -- returns larger chunks than naive retrieval
- [ ] Compared Parent vs naive: noted quality difference on 3+ queries
- [ ] SelfQueryRetriever filters by page number or source correctly
- [ ] EnsembleRetriever: tested with exact keyword query -- BM25 catches it
- [ ] HyDE implemented -- hypothetical doc used as retrieval query
- [ ] A/B test done on 20-question set -- best retriever identified and documented
- [ ] Can explain: why ParentDocumentRetriever helps, what BM25 adds over semantic

---

Day 19 . Chains & Routing -- RunnableBranch, Parallel, MapReduce

(1) THEORY - TOPICS TO COVER

RunnableBranch -- conditional routing: different chains for different inputs
RunnableParallel -- running multiple chains on the same input simultaneously
MapReduceDocumentsChain -- summarising docs too long for one context window
StuffDocumentsChain vs MapReduceChain -- tradeoffs for different doc sizes
LCEL router pattern: classifier chain -> route to specialist chain

(2) FOCUS / SKIP GUIDE

FOCUS ON

RunnableBranch: list of (condition_fn, chain) tuples plus a default fallback
RunnableParallel: dict syntax -- keys become keys in the output dict
Map step: each chunk processed independently; reduce step: combine all
Measuring execution time of parallel vs sequential -- seeing the actual speedup
The classifier-then-route pattern: one LLM call decides, another executes

DON'T WORRY ABOUT YET

MapRerankDocumentsChain -- niche pattern, rarely needed
Async parallel execution and asyncio internals -- focus on sync first
Custom chain classes from Chain base class -- LCEL replaces them
Streaming from parallel branches simultaneously

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LCEL routing cookbook
-> https://python.langchain.com/docs/how_to/routing/
[Docs] LCEL parallel how-to
-> https://python.langchain.com/docs/how_to/parallel/
[Docs] LangChain -- MapReduce summarisation
-> https://python.langchain.com/docs/how_to/summarize_map_reduce/
[Docs] LangChain -- Summarisation techniques overview
-> https://python.langchain.com/docs/tutorials/summarization/
[YouTube] LangChain -- Chains and routing tutorial
-> https://www.youtube.com/watch?v=ZHBbgMlWnFw

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Build RunnableBranch: coding -> coding chain, maths -> maths chain, else -> general
- [ ] Use RunnableParallel to get a brief AND detailed answer simultaneously
- [ ] Time parallel execution vs sequential -- print both times
- [ ] Build a map-reduce summariser for a PDF of 10+ pages
- [ ] Compare Stuff vs MapReduce on a 5-page and a 30-page document

TASK RESOURCES

[Docs] LangChain -- RunnableBranch API reference
-> https://python.langchain.com/api_reference/core/runnables/langchain_core.runnables.branch.RunnableBranch.html
[Docs] LangChain -- RunnableParallel API reference
-> https://python.langchain.com/api_reference/core/runnables/langchain_core.runnables.base.RunnableParallel.html
[Docs] LangChain -- create_map_reduce_documents_chain
-> https://python.langchain.com/docs/how_to/summarize_map_reduce/

(4) CODE EXAMPLES

RunnableBranch with 3 routes

```
from langchain_core.runnables import RunnableBranch
from langchain_core.output_parsers import StrOutputParser

coding_chain = ChatPromptTemplate.from_template('Answer this coding question: {input}') | llm | StrOutputPar
maths_chain  = ChatPromptTemplate.from_template('Solve step by step: {input}') | llm | StrOutputParser()
general_chain= ChatPromptTemplate.from_template('Answer helpfully: {input}') | llm | StrOutputParser()

def is_coding(x): return any(w in x['input'].lower() for w in ['python','code','function','bug','class'])
def is_maths(x):  return any(w in x['input'].lower() for w in ['calculate','solve','equation','integral'])

router = RunnableBranch(
    (is_coding, coding_chain),
    (is_maths,  maths_chain),
    general_chain,
)
print(router.invoke({'input': 'How do I write a Python decorator?'}))
```

RunnableParallel + MapReduce summariser

```
from langchain_core.runnables import RunnableParallel
import time

brief_chain   = ChatPromptTemplate.from_template('Explain {topic} in 1 sentence.') | llm | StrOutputParser()
detailed_chain= ChatPromptTemplate.from_template('Explain {topic} in 5 sentences.') | llm | StrOutputParser()

parallel = RunnableParallel(brief=brief_chain, detailed=detailed_chain)
start = time.time()
results = parallel.invoke({'topic': 'attention mechanism'})
print(f'Parallel: {time.time()-start:.2f}s')

# MapReduce summariser
from langchain.chains.summarize import load_summarize_chain
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

docs   = PyPDFLoader('long.pdf').load()
chunks = RecursiveCharacterTextSplitter(chunk_size=2000).split_documents(docs)
chain  = load_summarize_chain(llm, chain_type='map_reduce', verbose=True)
print(chain.invoke({'input_documents': chunks})['output_text'])
```

READ MORE - TOPIC REFERENCES

[Docs] LangChain -- Stuff summarisation (for short docs)
-> https://python.langchain.com/docs/how_to/summarize_stuff/
[Docs] LangChain -- Refine chain summarisation
-> https://python.langchain.com/docs/how_to/summarize_refine/

(5) DAY COMPLETION CHECKLIST

- [ ] RunnableBranch with 3 routes works -- each query goes to the right chain
- [ ] RunnableParallel runs two chains and returns both results in a dict
- [ ] Timed parallel vs sequential -- parallel is measurably faster
- [ ] Map-reduce summariser handles a 10+ page PDF without token errors
- [ ] Tested Stuff on short doc and MapReduce on long doc -- noted tradeoffs
- [ ] Can explain: when to use RunnableBranch vs manual if/else routing

---

Day 20-21 . Week 3 Project -- Research Assistant with Streaming

(1) THEORY - TOPICS TO COVER

LangChain callbacks system -- how it intercepts chain events
Streaming with LCEL: .stream() vs .astream() vs .astream_events()
Building a research assistant: search -> summarise -> route
Tavily API for live web search integration

(2) FOCUS / SKIP GUIDE

FOCUS ON

Combining a web search tool with your local RAG retriever via routing
Streaming with .stream() so tokens print as they arrive in CLI
Using LangSmith to trace the full research assistant and find bottlenecks
The routing decision: when to search the web vs query local docs
Putting all Week 3 skills together: tools + agents + retrieval + routing

DON'T WORRY ABOUT YET

Frontend streaming UI -- that is the FastAPI project in Week 7
Caching search results to save API costs -- optimisation for later
Rate limiting and retries for the search API -- production concern

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] Tavily search API (free tier) -- sign up
-> https://tavily.com/
[Docs] LangChain -- Tavily search integration
-> https://python.langchain.com/docs/integrations/tools/tavily_search/
[YouTube] Mervin Praison -- Research agent with LangChain
-> https://www.youtube.com/watch?v=g8DD5WsFgEk
[Docs] LangChain -- Streaming how-to guide
-> https://python.langchain.com/docs/how_to/streaming/

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Sign up for Tavily free tier and get an API key
- [ ] pip install tavily-python
- [ ] Build research assistant: takes a topic, searches web, summarises results
- [ ] Add routing: local docs questions -> RAG, recent/news questions -> web search
- [ ] Implement streaming output with .stream() -- tokens print in real time
- [ ] Trace the full run in LangSmith -- identify the slowest step
- [ ] Test with 5 research topics and review summary quality

TASK RESOURCES

[Docs] LangChain -- TavilySearchResults tool
-> https://python.langchain.com/docs/integrations/tools/tavily_search/
[Docs] LangChain -- How to stream agent steps
-> https://python.langchain.com/docs/how_to/streaming_agent/
[Docs] LangChain -- astream_events reference
-> https://python.langchain.com/docs/how_to/astream_events/

(4) CODE EXAMPLES

Tavily search tool + research agent

```
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain.agents import create_tool_calling_agent, AgentExecutor
import os

os.environ['TAVILY_API_KEY'] = 'tvly-...'
search_tool = TavilySearchResults(max_results=3)
tools = [search_tool, calculator, get_current_time]

research_prompt = ChatPromptTemplate.from_messages([
    ('system', '''You are a research assistant.
    Search for information, synthesise findings, highlight key points.'''),
    ('human', '{input}'),
    MessagesPlaceholder('agent_scratchpad'),
])
agent    = create_tool_calling_agent(llm, tools, research_prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

Streaming + routing: web vs local docs

```
# Stream tokens from chain
for chunk in chain.stream({'input': 'Explain transformers'}):
    print(chunk, end='', flush=True)

# Route: web search vs local RAG
from langchain_core.runnables import RunnableBranch, RunnableLambda

LOCAL_KW = ['document','pdf','uploaded','file','chapter','page']
WEB_KW   = ['latest','recent','today','2024','2025','news']

def needs_web(x):
    q = x['input'].lower()
    return any(w in q for w in WEB_KW) and not any(w in q for w in LOCAL_KW)

research_router = RunnableBranch(
    (needs_web, RunnableLambda(lambda x: executor.invoke(x)['output'])),
    rag_chain,
)
```

READ MORE - TOPIC REFERENCES

[Docs] LangChain -- DuckDuckGo search (free alternative)
-> https://python.langchain.com/docs/integrations/tools/ddg/
[Blog] LangChain -- Building research assistants
-> https://blog.langchain.dev/building-research-assistants/

(5) DAY COMPLETION CHECKLIST

- [ ] Tavily API key set up -- TavilySearchResults returns real web results
- [ ] Research assistant summarises findings from 3 web sources
- [ ] Routing works: web queries go to Tavily, local queries go to RAG retriever
- [ ] Streaming output working -- tokens print to terminal in real time
- [ ] LangSmith trace shows full run -- bottleneck step identified
- [ ] Tested 5 research topics -- quality of summaries reviewed and noted
- [ ] Week 3 COMPLETE -- all 6 days done, ready for LangGraph

Week 4 -- LangGraph Intro

Day 22 . Why LangGraph -- Graphs vs Chains

(1) THEORY - TOPICS TO COVER

Limitations of LCEL chains: no loops, no dynamic branching, no persistence
What a graph is: nodes (functions), edges (transitions), shared state dict
LangGraph mental model -- a state machine for AI workflows
When to use LangChain chains vs LangGraph graphs
Graph terminology: directed graph, cyclic graph, conditional edges

(2) FOCUS / SKIP GUIDE

FOCUS ON

The key difference: chains are linear pipelines; graphs can loop and branch
Why agents need loops -- the tool call -> result -> next decision cycle
State: the single shared dict that every node reads from and writes to
Why checkpointing is hard in chains but trivial in LangGraph
Drawing your existing RAG app as a graph -- which steps are nodes?

DON'T WORRY ABOUT YET

LangGraph Platform cloud deployment -- production topic for Weeks 7-8
Distributed multi-agent systems over networks -- advanced architecture
LangGraph Studio installation -- optional visual tool, not required
Async execution and astream patterns -- come back after sync basics

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangGraph -- Why LangGraph? (official conceptual page)
-> https://langchain-ai.github.io/langgraph/concepts/why_langgraph/
[Docs] LangGraph conceptual overview -- read this end to end
-> https://langchain-ai.github.io/langgraph/concepts/
[YouTube] LangChain -- LangGraph intro video
-> https://www.youtube.com/watch?v=R0XGQHL-4LM
[YouTube] Sam Witteveen -- LangGraph from scratch
-> https://www.youtube.com/watch?v=R8KB-Zcynxc
[Docs] LangGraph GitHub README
-> https://github.com/langchain-ai/langgraph

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] pip install langgraph -- confirm 'from langgraph.graph import StateGraph' works
- [ ] Read the LangGraph conceptual overview page fully
- [ ] Draw your Week 2 RAG app as a graph on paper -- nodes + edges
- [ ] List: what is the state? what are the nodes? what are the edges?
- [ ] Read the LangGraph quickstart and build the first hello-world StateGraph

TASK RESOURCES

[Docs] LangGraph quickstart tutorial (official)
-> https://langchain-ai.github.io/langgraph/tutorials/introduction/
[Docs] LangGraph -- Core concepts glossary
-> https://langchain-ai.github.io/langgraph/concepts/low_level/

(4) CODE EXAMPLES

First StateGraph -- hello LangGraph

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# 1. Define the shared state
class MyState(TypedDict):
    input:  str
    output: str
    count:  int

# 2. Define nodes (plain Python functions)
def process_node(state: MyState) -> dict:
    return {'output': state['input'].upper(), 'count': state.get('count', 0) + 1}

def review_node(state: MyState) -> dict:
    return {'output': f'Reviewed: {state["output"]}'}

# 3. Build the graph
graph = StateGraph(MyState)
graph.add_node('process', process_node)
graph.add_node('review',  review_node)
graph.add_edge(START,     'process')
graph.add_edge('process', 'review')
graph.add_edge('review',  END)

# 4. Compile and run
app    = graph.compile()
result = app.invoke({'input': 'hello langgraph', 'count': 0, 'output': ''})
print(result)  # {'input': ..., 'output': 'Reviewed: HELLO LANGGRAPH', 'count': 1}
```

READ MORE - TOPIC REFERENCES

[Blog] LangChain blog -- Intro to LangGraph
-> https://blog.langchain.dev/langgraph/

(5) DAY COMPLETION CHECKLIST

- [ ] langgraph installed -- import works without errors
- [ ] Read conceptual overview -- understand nodes, edges, state
- [ ] Drew the RAG app as a graph -- identified all nodes and edges
- [ ] First StateGraph runs -- .invoke() returns expected output
- [ ] Can explain: why chains cannot loop, and how graphs solve this
- [ ] Can explain: what shared state is and why every node uses it

Day 23 . StateGraph -- Nodes, Edges & Conditional Routing

(1) THEORY - TOPICS TO COVER

TypedDict for defining graph state -- every field must be declared
add_node(name, fn) -- registering functions as graph nodes
add_edge(from, to) -- unconditional transitions between nodes
add_conditional_edges(from, routing_fn, mapping) -- dynamic routing
START and END -- special built-in nodes in every graph
State updates: nodes return partial dicts, not the full state

(2) FOCUS / SKIP GUIDE

FOCUS ON

TypedDict: every field the graph uses must be declared upfront
Nodes return only the fields they update -- LangGraph merges into state
Conditional edges: routing fn returns a string key; mapping routes to node
compile() validates the graph and returns a runnable Pregel object
.invoke() vs .stream() -- stream shows each node output as it runs
Testing: print state after each node to confirm correctness

DON'T WORRY ABOUT YET

Graph reducers and custom merge logic (add_messages) -- Day 30 topic
Subgraphs nested inside parent graphs -- Week 5
Async node functions -- focus on sync for now
LangGraph low-level Pregel internals

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangGraph quickstart tutorial
-> https://langchain-ai.github.io/langgraph/tutorials/introduction/
[Docs] LangGraph -- Low-level concepts (StateGraph)
-> https://langchain-ai.github.io/langgraph/concepts/low_level/
[YouTube] Sam Witteveen -- LangGraph StateGraph deep dive
-> https://www.youtube.com/watch?v=R8KB-Zcynxc
[Docs] LangGraph how-to: conditional edges
-> https://langchain-ai.github.io/langgraph/how-tos/branching/

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Build graph: classify -> (coding OR general) -> END
- [ ] Define TypedDict state: question, question_type, answer fields
- [ ] Add a conditional edge on classify node output
- [ ] Compile and run with .invoke()
- [ ] Use .stream() -- print each node name and output as it runs
- [ ] Add a counter + loop: keep incrementing until count reaches 3, then END

TASK RESOURCES

[Docs] LangGraph -- StateGraph API reference
-> https://langchain-ai.github.io/langgraph/reference/graphs/
[Docs] LangGraph -- How to stream state updates
-> https://langchain-ai.github.io/langgraph/how-tos/stream-updates/
[Docs] LangGraph -- How to visualise your graph
-> https://langchain-ai.github.io/langgraph/how-tos/draw-graphs/

(4) CODE EXAMPLES

Conditional routing with TypedDict state

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Literal
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

class State(TypedDict):
    question:      str
    question_type: str
    answer:        str

def classify_node(state: State) -> dict:
    prompt = f'Is this a coding question? Answer yes or no only: {state["question"]}'
    resp = llm.invoke([HumanMessage(prompt)]).content.strip().lower()
    return {'question_type': 'coding' if 'yes' in resp else 'general'}

def coding_node(state: State) -> dict:
    resp = llm.invoke([HumanMessage(f'Answer with code: {state["question"]}')])
    return {'answer': resp.content}

def general_node(state: State) -> dict:
    resp = llm.invoke([HumanMessage(f'Answer concisely: {state["question"]}')])
    return {'answer': resp.content}

def route(state: State) -> Literal['coding', 'general']:
    return state['question_type']

graph = StateGraph(State)
graph.add_node('classify', classify_node)
graph.add_node('coding',   coding_node)
graph.add_node('general',  general_node)
graph.add_edge(START, 'classify')
graph.add_conditional_edges('classify', route, {'coding': 'coding', 'general': 'general'})
graph.add_edge('coding', END)
graph.add_edge('general', END)

app = graph.compile()
for step in app.stream({'question': 'How do I write a Python class?', 'question_type':'', 'answer':''}):
    print(step)
```

Visualise your graph

```
# Print ASCII in terminal
app.get_graph().print_ascii()

# Export as Mermaid -- paste at https://mermaid.live
print(app.get_graph().draw_mermaid())
```

READ MORE - TOPIC REFERENCES

[Docs] LangGraph -- MessagesState built-in state class
-> https://langchain-ai.github.io/langgraph/reference/graphs/#langgraph.graph.message.MessagesState
[Docs] LangGraph -- Node retry policies
-> https://langchain-ai.github.io/langgraph/how-tos/node-retries/

(5) DAY COMPLETION CHECKLIST

- [ ] 3-node graph with conditional routing built and runs correctly
- [ ] TypedDict state has all required fields declared
- [ ] Conditional edge routes to correct node based on classification result
- [ ] Graph compiled -- both .invoke() and .stream() work
- [ ] .stream() shows each node name and its state update dict
- [ ] ASCII and Mermaid diagrams generated successfully
- [ ] Can explain: why nodes return partial dicts, not the full state object

---

Day 24 . Building a ReAct Agent in LangGraph

(1) THEORY - TOPICS TO COVER

ReAct pattern as a looping graph -- not a linear chain
The agent node: call LLM -> check if tool_calls exist in response
The tool node: execute tools -> return ToolMessage results
Loop condition: tool_calls exist -> tool node, else -> END
MessagesState -- built-in state with add_messages reducer (appends)
tools_condition -- prebuilt conditional edge for tool routing

(2) FOCUS / SKIP GUIDE

FOCUS ON

Why this is a graph: it loops until the LLM stops calling tools
MessagesState: messages field appends (not replaces) on each update
Two-node structure: agent_node <-> tools_node with conditional edge
ToolNode prebuilt: runs any tool the LLM calls automatically
tools_condition: returns 'tools' if tool_calls exist, else END
Print all messages in final state to trace the full loop

DON'T WORRY ABOUT YET

Human-in-the-loop interrupts -- Day 26
Checkpointing and persistence -- Day 25
Multi-agent orchestration -- Week 5
Custom tool node implementations -- ToolNode covers most cases

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangGraph ReAct agent tutorial (official)
-> https://langchain-ai.github.io/langgraph/tutorials/introduction/
[Docs] LangGraph -- Build a ReAct agent (official how-to)
-> https://langchain-ai.github.io/langgraph/how-tos/create-react-agent/
[YouTube] LangChain -- Build a ReAct agent with LangGraph
-> https://www.youtube.com/watch?v=hvAPnpSfSGo
[Docs] LangGraph -- prebuilt ToolNode reference
-> https://langchain-ai.github.io/langgraph/reference/prebuilt/

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Build ReAct agent graph: define agent_node and tool_node manually
- [ ] Add conditional edge: tool_calls -> tools, no tool_calls -> END
- [ ] Test with a query requiring 1 tool call
- [ ] Test with a query requiring 2 sequential tool calls
- [ ] Print all messages in final state -- read the full trace
- [ ] Replace manual tool_node with prebuilt ToolNode -- confirm same behaviour
- [ ] Try create_react_agent prebuilt -- same result in one line

TASK RESOURCES

[Docs] LangGraph -- MessagesState reference
-> https://langchain-ai.github.io/langgraph/reference/graphs/#langgraph.graph.message.MessagesState
[Docs] LangGraph -- tools_condition reference
-> https://langchain-ai.github.io/langgraph/reference/prebuilt/#langgraph.prebuilt.tool_node.tools_condition
[Docs] LangGraph -- ToolNode reference
-> https://langchain-ai.github.io/langgraph/reference/prebuilt/#langgraph.prebuilt.tool_node.ToolNode

(4) CODE EXAMPLES

ReAct agent from scratch

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
tools = [calculator, get_current_time, search_wikipedia]
llm_with_tools = llm.bind_tools(tools)

def agent_node(state: MessagesState) -> dict:
    response = llm_with_tools.invoke(state['messages'])
    return {'messages': [response]}  # add_messages appends

graph = StateGraph(MessagesState)
graph.add_node('agent', agent_node)
graph.add_node('tools', ToolNode(tools))
graph.add_edge(START, 'agent')
graph.add_conditional_edges('agent', tools_condition)
graph.add_edge('tools', 'agent')  # loop back

app = graph.compile()
result = app.invoke({'messages': [HumanMessage('What is sqrt(1764) and what time is it?')]})

for msg in result['messages']:
    print(f'{type(msg).__name__}: {str(msg.content)[:80]}')
```

create_react_agent prebuilt shortcut

```
from langgraph.prebuilt import create_react_agent

# Builds the same agent + tool + loop graph in one line
app = create_react_agent(llm, tools=tools)

result = app.invoke({
    'messages': [HumanMessage('Search Wikipedia for Marie Curie')]
})
print(result['messages'][-1].content)
```

READ MORE - TOPIC REFERENCES

[Docs] LangGraph -- Handle tool call errors in graph
-> https://langchain-ai.github.io/langgraph/how-tos/tool-calling-errors/
[Paper] ReAct paper -- Yao et al. 2022
-> https://arxiv.org/abs/2210.03629

(5) DAY COMPLETION CHECKLIST

- [ ] agent_node calls LLM with bind_tools and returns AIMessage
- [ ] Manual tool_node built first -- then replaced with ToolNode prebuilt
- [ ] Conditional edge routes to tools if tool_calls exist, else END
- [ ] Loop edge tools -> agent confirmed -- graph loops until no more tools
- [ ] 1-tool-call query works correctly
- [ ] 2-sequential-tool-call query works correctly
- [ ] Full message trace printed -- can read the full loop history

- [ ] create_react_agent prebuilt produces same results in one line

Day 25 . Checkpointing & Persistence

(1) THEORY - TOPICS TO COVER

Checkpointing -- saving full graph state after each node executes
MemorySaver -- in-memory checkpointing (lost when Python exits)
SqliteSaver -- SQLite-backed persistence (survives restarts)
Thread IDs -- how LangGraph separates multiple conversations
config = {'configurable': {'thread_id': ...}} -- how to pass thread_id
Time-travel debugging -- replaying execution from an earlier checkpoint

(2) FOCUS / SKIP GUIDE

FOCUS ON

The config dict pattern: {'configurable': {'thread_id': 'user_1'}}
Thread IDs let you run many independent conversations on one compiled graph
MemorySaver for development; SqliteSaver for anything needing persistence
How to confirm SqliteSaver works: restart Python -> invoke -> same memory
app.get_state(config) to inspect the saved state for any thread

DON'T WORRY ABOUT YET

PostgresSaver -- production database persistence, same API as SqliteSaver
State schema migration when you change TypedDict fields
Checkpoint pruning and storage limits at scale
Distributed checkpoint stores across multiple machines

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangGraph persistence concepts
-> https://langchain-ai.github.io/langgraph/concepts/persistence/
[Docs] LangGraph checkpointers reference
-> https://langchain-ai.github.io/langgraph/reference/checkpoints/
[Docs] LangGraph -- How to add persistence
-> https://langchain-ai.github.io/langgraph/how-tos/persistence/
[YouTube] LangChain -- LangGraph memory tutorial
-> https://www.youtube.com/watch?v=R8KB-Zcynxc

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Add MemorySaver to your Day 24 ReAct agent
- [ ] Run agent twice with same thread_id -- confirm it remembers first message
- [ ] Run with a different thread_id -- confirm completely separate memory
- [ ] Switch to SqliteSaver -- run agent, restart Python, run again -- confirm memory
- [ ] Use app.get_state(config) to inspect the saved state for a thread
- [ ] Call app.get_state_history(config) -- see list of checkpoint snapshots

TASK RESOURCES

[Docs] LangGraph -- MemorySaver reference
-> https://langchain-ai.github.io/langgraph/reference/checkpoints/#langgraph.checkpoint.memory.MemorySaver
[Docs] LangGraph -- SqliteSaver reference
-> https://langchain-ai.github.io/langgraph/reference/checkpoints/#langgraph.checkpoint.sqlite.SqliteSaver
[Docs] LangGraph -- How to view and update graph state
-> https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/

(4) CODE EXAMPLES

MemorySaver -- multi-turn memory per thread

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.prebuilt import create_react_agent
from langchain_core.messages import HumanMessage

memory = MemorySaver()
app    = create_react_agent(llm, tools=tools, checkpointer=memory)

cfg_alice = {'configurable': {'thread_id': 'alice'}}
app.invoke({'messages': [HumanMessage('My name is Alice')]}, config=cfg_alice)
resp = app.invoke({'messages': [HumanMessage('What is my name?')]}, config=cfg_alice)
print(resp['messages'][-1].content)  # 'Your name is Alice'

# Different thread -- no memory of Alice
cfg_bob = {'configurable': {'thread_id': 'bob'}}
resp2 = app.invoke({'messages': [HumanMessage('What is my name?')]}, config=cfg_bob)
print(resp2['messages'][-1].content)  # does not know
```

SqliteSaver -- survives Python restarts

```
from langgraph.checkpoint.sqlite import SqliteSaver

with SqliteSaver.from_conn_string('agent_memory.db') as checkpointer:
    app = create_react_agent(llm, tools=tools, checkpointer=checkpointer)
    cfg = {'configurable': {'thread_id': 'persistent_user'}}

    # First run
    app.invoke({'messages': [HumanMessage('I love Python')]}, config=cfg)

    # Inspect state
    state = app.get_state(cfg)
    print(f'Messages saved: {len(state.values["messages"])}')

    # Restart Python, re-run the with block, then:
    resp = app.invoke({'messages': [HumanMessage('What do I love?')]}, config=cfg)
    print(resp['messages'][-1].content)  # 'You love Python'
```

READ MORE - TOPIC REFERENCES

[Docs] LangGraph -- Time travel: replay from checkpoints
-> https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/time-travel/
[Docs] LangGraph -- PostgresSaver for production
-> https://langchain-ai.github.io/langgraph/reference/checkpoints/#langgraph.checkpoint.postgres.aio.AsyncPostgresSaver

(5) DAY COMPLETION CHECKLIST

- [ ] MemorySaver added -- same thread_id maintains memory across calls
- [ ] Two different thread_ids have completely separate memories -- verified
- [ ] SqliteSaver works -- state persists to agent_memory.db file
- [ ] Python restarted -- SqliteSaver agent still remembers previous conversation
- [ ] app.get_state(config) used -- inspected messages list in saved state
- [ ] app.get_state_history(config) called -- saw list of checkpoint snapshots
- [ ] Can explain: what a thread_id is and why it matters for multi-user apps

---

Day 26 . Human-in-the-Loop -- Interrupts & State Updates

(1) THEORY - TOPICS TO COVER

interrupt_before=['node'] -- pause graph BEFORE a node executes
interrupt_after=['node'] -- pause graph AFTER a node executes
Command(resume=value) -- resuming a paused graph with a decision
graph.update_state(config, values) -- injecting changes into paused graph
Use cases: approval workflows, human confirmation before tools fire
The interrupt mechanism -- how LangGraph stores the pause point

(2) FOCUS / SKIP GUIDE

FOCUS ON

interrupt_before: node has NOT run yet -- you can cancel or modify input
interrupt_after: node HAS run -- you can modify or override the output
Two-step invoke pattern: first invoke pauses; second invoke with Command resumes
update_state lets you inject a human message into the graph mid-execution
Testing: verify graph pauses, prints tool details, resumes correctly

DON'T WORRY ABOUT YET

Complex approval UIs or web interfaces -- CLI testing is enough for now
Nested interrupt chains across multiple nodes simultaneously
Interrupt handling in async execution contexts
Production interrupt queuing with databases

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangGraph human-in-the-loop concepts
-> https://langchain-ai.github.io/langgraph/concepts/human_in_the_loop/
[Docs] LangGraph -- How to add breakpoints
-> https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/breakpoints/
[Docs] LangGraph -- How to edit graph state
-> https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/edit-graph-state/
[YouTube] LangChain -- Human-in-the-loop with LangGraph
-> https://www.youtube.com/watch?v=9BPCV5TYPmg

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Add interrupt_before=['tools'] to your Day 24 ReAct agent
- [ ] Run a query -- verify graph pauses and shows tool name + args

- [ ] Resume with Command(resume=None) -- verify tool runs and agent completes
- [ ] Test rejection: cancel the tool call -- observe agent reroutes
- [ ] Use graph.update_state() to inject a human correction message
- [ ] Build review-and-edit workflow: agent drafts -> human edits -> agent finalises

TASK RESOURCES

[Docs] LangGraph -- Command object reference
-> https://langchain-ai.github.io/langgraph/reference/types/#langgraph.types.Command
[Docs] LangGraph -- update_state reference
-> https://langchain-ai.github.io/langgraph/reference/graphs/#langgraph.graph.graph.CompiledGraph.update_state
[Docs] LangGraph -- How to wait for user input
-> https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/wait-user-input/

(4) CODE EXAMPLES

interrupt_before -- approve/reject tool calls

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import Command
from langgraph.prebuilt import create_react_agent
from langchain_core.messages import HumanMessage

memory = MemorySaver()
app = create_react_agent(
    llm, tools=tools, checkpointer=memory,
    interrupt_before=['tools'],
)
cfg = {'configurable': {'thread_id': 'approval_demo'}}

# Step 1: invoke -- graph pauses before tools node
app.invoke({'messages': [HumanMessage('What is 999*888?')]}, config=cfg)

# Print what tool the agent wants to call
state = app.get_state(cfg)
last  = state.values['messages'][-1]
if last.tool_calls:
    print(f'Tool: {last.tool_calls[0]["name"]}')
    print(f'Args: {last.tool_calls[0]["args"]}')

# Step 2: human decision
decision = input('Approve? (y/n): ')
if decision.lower() == 'y':
    final = app.invoke(Command(resume=None), config=cfg)
    print(final['messages'][-1].content)
```

update_state -- inject human feedback

```
from langchain_core.messages import HumanMessage

# After agent produces output, inject a correction
correction = HumanMessage(
    content='Please use simpler language in your answer.'
)
app.update_state(
    cfg,
    {'messages': [correction]},
    as_node='agent'
)

# Resume from updated state
final = app.invoke(Command(resume=None), config=cfg)
print(final['messages'][-1].content)
```

READ MORE - TOPIC REFERENCES

[Docs] LangGraph -- Dynamic breakpoints with interrupt()
-> https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/dynamic_breakpoints/
[Docs] LangGraph -- How to review tool calls
-> https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/review-tool-calls/

(5) DAY COMPLETION CHECKLIST

- [ ] interrupt_before=['tools'] added -- graph pauses before tool execution
- [ ] Tool call details printed when paused -- name and args visible
- [ ] Approval: Command(resume=None) resumes and agent completes correctly
- [ ] Rejection: tool is not run and agent reroutes gracefully
- [ ] update_state used -- human correction injected into running graph
- [ ] Review-and-edit workflow working end-to-end
- [ ] Can explain: interrupt_before vs interrupt_after and when to use each

---

Day 27-28 . Week 4 Project -- Stateful Agent with Persistence & Streaming

(1) THEORY - TOPICS TO COVER

LangGraph Studio overview -- live graph visualisation during runs
LangGraph streaming: stream_mode options (values, updates, debug)
Mermaid diagram export with get_graph().draw_mermaid()
Combining all Week 4 features: StateGraph + checkpointing + interrupts

(2) FOCUS / SKIP GUIDE

FOCUS ON

Rebuilding the Week 1 Q&A; bot as a full LangGraph agent with all features
SqliteSaver: every conversation persists across Python restarts
interrupt_before the tools node -- human reviews before tool fires
.stream(stream_mode='updates') -- prints each node output live
Exporting and reading the Mermaid diagram of your full graph

DON'T WORRY ABOUT YET

LangGraph Studio installation issues -- use Mermaid export instead
Making the agent available via API -- that is the Week 7 FastAPI project
Multi-agent supervisor systems -- Week 5

RESOURCES - WATCH / READ BEFORE STARTING

[Docs] LangGraph streaming guide
-> https://langchain-ai.github.io/langgraph/how-tos/streaming/
[Docs] LangGraph -- How to stream from a graph
-> https://langchain-ai.github.io/langgraph/how-tos/streaming-tokens/
[Docs] LangGraph Studio docs
-> https://langchain-ai.github.io/langgraph/concepts/langgraph_studio/
[YouTube] LangChain -- LangGraph Studio demo
-> https://www.youtube.com/watch?v=3Y4dBqsNOxs
[Tool] Mermaid Live Editor -- paste diagram code here
-> https://mermaid.live

(3) PRACTICAL - TASKS TO COMPLETE

- [ ] Rebuild Week 1 Q&A; bot as full LangGraph StateGraph agent
- [ ] Add a search_documents RAG tool using your Week 2 Chroma vectorstore
- [ ] Add SqliteSaver -- all conversations persist across restarts
- [ ] Add interrupt_before=['tools'] -- human approves tool calls
- [ ] Use .stream(stream_mode='updates') -- print each node as it completes
- [ ] Export Mermaid diagram -- visualise at mermaid.live
- [ ] Test 10 questions across 3 different thread IDs -- confirm all independent

TASK RESOURCES

[Docs] LangGraph -- stream_mode options
-> https://langchain-ai.github.io/langgraph/how-tos/streaming/
[Docs] LangGraph -- draw_mermaid reference
-> https://langchain-ai.github.io/langgraph/how-tos/draw-graphs/
[Docs] LangGraph Studio install guide
-> https://github.com/langchain-ai/langgraph-studio

(4) CODE EXAMPLES

Full stateful Q&A agent with all Week 4 features

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage

@tool
def search_documents(query: str) -> str:
    '''Search the uploaded documents for relevant information.'''
    docs = retriever.invoke(query)  # your Week 2 Chroma retriever
    return chr(10).join(d.page_content for d in docs[:3])

all_tools = [search_documents, calculator, get_current_time]
llm_with_tools = llm.bind_tools(all_tools)

def agent_node(state: MessagesState) -> dict:
    return {'messages': [llm_with_tools.invoke(state['messages'])]}

graph = StateGraph(MessagesState)
graph.add_node('agent', agent_node)
graph.add_node('tools', ToolNode(all_tools))
graph.add_edge(START, 'agent')
graph.add_conditional_edges('agent', tools_condition)
graph.add_edge('tools', 'agent')

with SqliteSaver.from_conn_string('qa_agent.db') as ckpt:
    app = graph.compile(checkpointer=ckpt, interrupt_before=['tools'])
    cfg = {'configurable': {'thread_id': 'session_1'}}
    for update in app.stream(
        {'messages': [HumanMessage('What does the doc say about RAG?')]},
        config=cfg, stream_mode='updates'
    ):
        print(update)
```

Export Mermaid diagram

```
# Mermaid code for mermaid.live
print(app.get_graph().draw_mermaid())

# Save to file
with open('agent_graph.md', 'w') as f:
    f.write('```mermaid\n' + app.get_graph().draw_mermaid() + '```')

# Or print ASCII in terminal
app.get_graph().print_ascii()
```

READ MORE - TOPIC REFERENCES

[Docs] LangGraph -- Streaming tokens from LLM nodes
-> https://langchain-ai.github.io/langgraph/how-tos/streaming-tokens/
[Blog] LangChain -- Building production agents with LangGraph
-> https://blog.langchain.dev/langgraph-for-production/

(5) DAY COMPLETION CHECKLIST

- [ ] Q&A; bot rebuilt as full LangGraph StateGraph
- [ ] search_documents RAG tool connected to Week 2 Chroma vectorstore

- [ ] SqliteSaver active -- conversations persist across Python restarts
- [ ] interrupt_before working -- human approval before tool execution
- [ ] .stream(stream_mode='updates') shows each node output as it runs
- [ ] Mermaid diagram exported and visualised at mermaid.live
- [ ] Tested 10 questions across 3 thread IDs -- all independent and correct
- [ ] WEEK 4 COMPLETE -- ready for Week 5 multi-agent systems

---